Operating system

Orange level problem

Name :Ankith Sudala

SRN : PES1UG24CS557

Section : 4J

Phase 1:

Screenshot 1A:

```
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B:

```
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ find .pes/objects -type f

.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566
fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f
476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaf
c12
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$
```

Phase 2:

Screenshot 2A:

```
      |     ^~~~~~~~~~~~~~~~~~~~
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$
```

Screenshot 2B:

```
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ xxd .pes/objects/2a/594d39232
787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$
```

Phase 3:

Screenshot 3A:

```
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ echo "hello" > file1.txt
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ echo "world" > file2.txt
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ ./pes add file1.txt file2.txt
Segmentation fault (core dumped)
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ ./pes status
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  index.c
  untracked:  test_tree
  untracked:  pes.c
  untracked:  test_objects
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  file1.txt
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  file2.txt
  untracked:  README.md
  untracked:  tree.h
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  object.c

anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$
```

Screenshot 3B:

```
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 177
6429826 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 177
6429826 6 file2.txt
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$
```

Phase 4:

Screenshot 4A:

```
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ ./pes log
commit cb19909b09509b45c2e7d2a054e09df62ae6dcb88cd7ac750c23b1221b2d24a5
Author: PES User <pes@localhost>
Date:   1776430697

    Add farewell

commit 0fcba6750bb176b1d4649e8fc2140ee6b37773ed109663d9767ba440afcbd1b4
Author: PES User <pes@localhost>
Date:   1776430678

    Add world

commit 4cbba14e4af0f52a01fe67ca940c03c05e72dbd9a4a99eaf80e3c43213e01d54
Author: PES User <pes@localhost>
Date:   1776430662

    Initial commit

anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$
```

Screenshot 4B:

```
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5
f43
.pes/objects/0f/cba6750bb176b1d4649e8fc2140ee6b37773ed109663d9767ba440afcbd
1b4
.pes/objects/4c/bba14e4af0f52a01fe67ca940c03c05e72dbd9a4a99eaf80e3c43213e01
d54
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5
c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a03
0f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626
442
.pes/objects/cb/19909b09509b45c2e7d2a054e09df62ae6dcb88cd7ac750c23b1221b2d2
4a5
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde
814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a6865
2bc
.pes/refs/heads/main
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$
```

Screenshot 4C:

```
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ cat .pes/refs/heads/main
cb19909b09509b45c2e7d2a054e09df62ae6dcb88cd7ac750c23b1221b2d24a5
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
anku@anku:~/os-orange2/PES1UG24CS557-pes-vcs$
```

Phase 5:

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Answer: To implement `pes checkout <branch>`, first check whether the branch file exists in `.pes/refs/heads/<branch>`. Then update `.pes/HEAD` so that it points to the selected branch. Read the commit hash stored in the branch file and load the snapshot of that commit. The working directory must then be updated to match that snapshot by creating new files, deleting removed files, and replacing modified files. If `.pes/` contains an index or staging metadata, it should also be updated. This operation is complex because uncommitted local changes may be overwritten, branches may have different file structures, and the working directory plus repository metadata must remain consistent.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer: To detect this conflict, compare three versions of each tracked file: the version in the index, the version in the current working directory, and the version in the target branch's tree. First, use the index as the record of the last known committed state of the working directory. For each tracked file, hash the current working directory file and compare its hash with the hash stored in the index. If the hashes are different, that file has uncommitted changes and the working directory is dirty for that file. Then look up the same file in the target branch's tree from the object store. If the target branch's version of that file has a different hash from the index version, then checkout would need to overwrite that file. If both conditions are true — the working file differs from the index, and the target branch's file differs from the index — then there is a dirty working directory conflict, and checkout must refuse. This works using only the index and object store because the index tells what is currently tracked, and the object store provides the file contents for the target branch.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer: In a detached HEAD state, new commits are created normally, but they are not added to any branch. HEAD simply moves from one commit to the next commit directly. This means the commits exist in the object store, but no branch name points to them. If the user later checks out another branch, those commits can become unreachable and may eventually be garbage-collected. To recover them, the user should create a new branch pointing to the detached HEAD commit before leaving that state, or later find the commit hash using reflog or recent commit history and then create a branch at that commit. This makes the commits reachable again and prevents them from being lost.

Phase 6:

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer:A simple algorithm is a mark-and-sweep garbage collection. First, start from all roots of the repository, such as the hashes stored in all branch references and HEAD. Put these commit hashes into a worklist such as a stack or queue. Then repeatedly remove one hash from the worklist, mark it as reachable, load its object from the object store, and inspect its references. If it is a commit, add its parent commit hashes and tree hash to the worklist. If it is a tree, add all referenced subtree and blob hashes. If it is a blob, nothing more needs to be

followed. Continue until the worklist is empty. After this mark phase, scan all objects in the object store. Any object whose hash is not marked reachable is unreachable and can be deleted.The best data structure to track reachable hashes efficiently is a hash set, because lookup and insertion are approximately O(1) on average. This lets us quickly check whether an object was already visited and avoid visiting the same object multiple times.For a repository with 100,000 commits and 50 branches, you would start from 50 branch tips, but many branches will share history, so you would not visit 50 × 100,000 commits. In the worst realistic case, you would visit about 100,000 unique commits, plus the trees and blobs reachable from them. So the total number of visited objects would usually be at least 100,000 commits, and in practice several times more after including tree and blob objects.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer: Running garbage collection at the same time as a commit operation is dangerous because GC may treat newly created objects as unreachable before the commit finishes updating references. During a commit, Git first writes new blobs and trees into the object store, then creates the commit object, and finally updates the branch reference to point to that new commit. Until the branch reference is updated, these new objects are temporarily unreferenced.A race condition can happen like this: a user starts a commit and new blob/tree objects are written. Before the branch reference is updated, garbage collection starts scanning reachable objects from branches and HEAD. Since the new objects are not yet referenced, GC marks them as unreachable and deletes them. Then the commit process tries to create or update the branch to reference those deleted objects, causing repository corruption or a broken commit.Git’s real garbage collection avoids this by using safety mechanisms such as locking references, keeping recently created loose objects for a grace period, and not immediately deleting unreachable objects. Git also uses reflogs and prune expiration times so that objects are only removed after they have remained unreachable for some time. This prevents deletion of objects that may still be part of an in-progress operation.